

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет
Лабораторная работа 2

Выполнила:
Олеся Долбенко
Группа: j3213

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г

Задача

Реализовать взаимодействие сайта с внешним api на базе json server:

- настроить локальный сервер, который хранит данные о пользователях, заметках, бюджете и сохранённых маршрутах
- сделать обработку ошибок при запросах к серверу
- проверка данных при входе и регистрации через сервер
- сделать выход из аккаунта с очисткой данных

Ход работы

В структуру проекта добавила:

- db.json - база данных
- script.js - логика работы с апи
- все html-страницы обновлены для работы с сервером

Этап 1: Настройка апи

Структура db.json:

- users - хранит данные пользователей (id, имя, email, пароль)
- destinations - хранит направления (название, тип, бюджет, длительность, описание, цена, фото, ссылка на карту)
- notes - хранит заметки пользователей (id, userId, заголовок, текст, дата создания)
- budgets - хранит бюджет пользователей
- savedRoutes - хранит сохранённые маршруты

Этап 2: Реализация авторизации через Fetch API

Написала функцию входа, которая обрабатывает форму входа.

Логика работы:

1. Считываю данные из формы (email, пароль)
2. Отправляю запрос к серверу
3. Получаю массив пользователей и ищу совпадение по почте и паролю
4. Если пользователь найден - сохраняю данные в localStorage (текущий пользователь) и перенаправляем на страницу личного кабинета
5. Если нет - ошибка «Неверный email или пароль»

Этап 3: Регистрации новых пользователей

Функция регистрации, которая создаёт нового пользователя через апи

Логика работы:

1. Проверяю, что все поля заполнены и пароли совпадают
2. Отправляю запрос, чтобы проверить, нет ли уже пользователя с таким email
3. Если email свободен – отправляю запрос с данными нового пользователя
4. После успешной регистрации отправляю на страницу входа

Этап 4: Выход из аккаунта

Функция `logout()`:

Логика работы:

1. Удаляем из `localStorage` данные текущего пользователя
2. Сбрасываем переменную `currentUser`
3. Сообщение о выходе
4. Отправляем на главную страницу

Этап 5: Защита маршрутов (перенаправление)

Реализована проверка авторизации на всех страницах:

Логика работы:

- Если пользователь не авторизован и пытается зайти в личный кабинет - «Доступ запрещён» + ссылка на страницу входа
- Если пользователь уже авторизован и пытается зайти на страницу входа или регистрации - автоматически отправляем в личный кабинет

Вывод

В ходе лабораторной работы я подключила `json server`, чтобы сайт мог хранить данные пользователей в отдельном файле. `fetch` ом обращаемся к серверу: когда пользователь входит - сайт проверяет email и пароль через запрос к серверу, при регистрации - добавляет нового пользователя в базу данных.

Сделала защиту личного кабинета: неавторизованные пользователи не могут попасть в профиль и перенаправляются на страницу входа.

В итоге на сайте работают регистрация, вход, выход, защита страниц. Если понадобится, его можно будет дальше дорабатывать, например, подключить настоящую базу данных или добавить сохранение заметок в api.